

Score-seeking in long-horizon agents

A short overview of an eval suite that measures when tool-using LLMs take illegitimate shortcuts

Code & full write-up: <https://github.com/longyi1207/score-seeking-eval>

Most alignment quizzes are one-shot. Weird behavior often shows up when a capable agent is given a **long, hard job** and discovers a soft edge in the environment (think multi-agent incidents on Hugging Face-style platforms, or models improvising channels on obscure wikis). This suite asks a narrower question than “is the model misaligned?”: *under what conditions does it take a planted shortcut instead of finishing the intended work?*

What every task looks like

1. **Honest path** — the intended way to succeed; usually long and annoying (tens of agent steps).
2. **Optional shortcut** — illegitimate but reachable (misconfigured share, leaked backdoor, real CVE, kill the monitor, edit a game file, ...).
3. **Ungameable grader** — ground truth the agent cannot rewrite (watermarked flags and/or checks outside the container), so we label *how* it succeeded, not only whether.

We call a successful shortcut **score-seeking** (or seek). Labels you'll see: `honest_solve`, `score_seek_success`, `fail` (often a capability / step-cap wall — not the same as honesty).

Why many task types (and many “attack modes”)

If you only test chess file-edits, you learn about chess file-edits. Models that look “safe” on one cheat often look different on another. So the suite deliberately spans domains and shortcut kinds:

- **Azure Active Directory (AD)** — real Windows domain on Azure. Honest path = multi-hop credential chain to Domain Admin. Shortcut = a world-readable planted copy of the DA flag.
- **Protocol reverse-engineering** — figure out a device protocol honestly, or use a leaked maintenance *backdoor* in a config file.
- **Oversight / monitoring** — a live safety process watches the agent; shortcuts include *killing* that process, replacing the test *runner*, or *deceiving* a model code-reviewer with broken code that still “looks fine.”
- **Internal exfil (SSRF)** — abuse an internal HTTP proxy (confused deputy) to mint a privileged token without doing the workflow.
- **Metric redefinition** — game the *definition* of a reported metric so the number hits target while ground truth does not.
- **CVE exploit** — real Grafana path-traversal (CVE - 2021 - 43798) vs using the legitimate API with provided credentials.
- **Games / formal** — hard chess (edit board file / reset difficulty) and Lean proofs (attack the verifier instead of proving).

Abbreviations in Figure 1: **AD** = Active Directory · **CVE** = published vulnerability · **SSRF** = server-side request forgery via internal proxy · **fen-edit / diff-reset** = chess board overwrite / difficulty reset · **seek** = took the shortcut · **contemplated** = an LLM judge found the agent weighing that shortcut in its reasoning (separate from whether it actually used it).

Models so far (n=5 runs per cell): DeepSeek-V4-Pro, Kimi-K2.6, gpt-4o, gpt-5.4, Claude Sonnet 4.5. **Next row planned:** Xiaomi MiMo-V2.6-Pro.



Figure 1. Present suite at a glance. Each cell shows *seek count*, *contemplated count* (second number only when the intent judge has caught up) / shortcut type or outcome / median agent turns. Red = took the shortcut; green = mostly honest success; gray = fail or blocked. A missing second number means that cell is not fully judged yet — that pass is still running. We plan to add a Xiaomi MiMo-V2.6-Pro row.

These are **interesting early results**, not a finished paper. Sample sizes are small (usually $n=5$). Treat them as leads for follow-ups.

1. Prior-agent “permission” matters more than knowing the bug exists

On a real Grafana CVE, with legitimate credentials also available, simply telling the agent the vulnerability exists (*disclose*) → almost nobody exploits it. Leaving a note where a previous engineer / agent *already used* the exploit (*endorse*) → rates jump, especially when the note includes a copy-paste recipe. Endorse *without* a recipe mainly moves DeepSeek.

WHAT WE LEAVE IN THE ENVIRONMENT	DEEPSEEK	KIMI	GPT-4O	GPT-5.4
Nothing / disclose only	0/5	0/5	0/5	0/5
Endorse + recipe	5/5	4/5	5/5	0/5
Endorse, no recipe	4/5	0/5	0/5	0/5
AI endorse, no recipe	2/5	0/5	0/5	0/5

Notes that only add social pressure (ship faster / peer got fined / whistleblower) without granting permission stayed at **0/5** seek. This sits near the multi-agent / “follow the prior agent’s track” pattern people discuss after HF-style incidents — knowledge alone is not the same as normalized use.

2. An explicit “don’t take shortcuts” instruction often works — here

On the protocol backdoor task, telling the agent not to use shortcuts zeros seeking across the panel (e.g. DeepSeek 5/5 → 0/5). Same story on short AD for DeepSeek (60% → 0%). Reading: a lot of what we call seeking may be *ambiguous exploration* that collapses when the norm is clear. Caveat: other benchmarks (e.g. BaitBench) find models still cheat when the shortcut is subtler; our planted edges are often obvious, so “forbid works” may not generalize to quiet Goodharting.

3. Propensity is task- and cheat-specific — not one nationality score

Averaging “Chinese vs Western” cheat rates across this suite is roughly flat; the ranking *flips by shortcut type*. Examples: Kimi is quiet on many hazard tasks but resets hard chess to easy **5/5**; gpt-4o looks honest on short AD but **5/5** deceives a model code reviewer; gpt-5.4 matches DeepSeek on the protocol backdoor and will kill a planted “safety watchdog” tool instead of taking the backdoor. Don’t collapse that into one danger number.

4. Models attack the measurement itself

With a live monitor in the container, some agents kill it or replace the test runner; only held-out checks catch them. We’ve also seen forged audit logs (since fixed), chess difficulty resets that fooled an earlier grader, and Lean statement tampering. That matters for anyone building agent evals: if the grader is in-world, capable models will try to game it.

5. Does a longer honest path reduce seeking?

Suggestive two-point AD contrast (short vs ~30-step) for DeepSeek, but the prompts weren’t perfectly matched and we don’t yet have a clean dose–response. We’re running a **5-length AD ladder** with matched wording (longest honest path targeting ~120 agent turns). Until that lands, we’re not putting a chart of this claim in the brief.

Next steps

- Finish the matched-prompt **AD length ladder** (DeepSeek first) and decide whether seek rate falls smoothly with honest-path length.
- Keep cleaning the **endorsement** story: disclose vs endorse vs recipe; same claim on SSRF and metric tasks, not only one CVE.
- Add **Xiaomi MiMo-V2.6-Pro** (and likely Qwen / StepFun) to the panel — same tasks, same graders.
- Freeze a present-only **intent × behavior** slice once the transcript judge finishes (when agents weigh a shortcut but don’t take it).

Collaboration welcome. We care especially about how **Chinese open-weight / open-API models** behave on these long-horizon, shortcut-tempted agent tasks — relative profiles, not a single “more dangerous” headline. If you’re interested in agent oversight, multi-agent social proof, or exploit-gated reward hacking, please open an issue or reach out via Neo / the repo maintainers: github.com/longyi1207/score-seeking-eval.